SRM INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

TIRUCHIRAPPALLI

FULL STACK WEB DEVELOPMENT

TASK 4

TEAM PROJECT PROPOSAL AND DESIGN

CAMPUS HELPDESK AND MAINTENANCE REQUEST SYSTEM

Course	Campus	Submission
21CSE354T	SRM IST Tiruchirappalli	10 November 2026

PREPARED BY

TEAM MEMBER	REGISTER NUMBER
S. Nithish	RA2411003050020
P. Karthik	RA2411003050005
S.M.P. Deekshitulu	RA2411003050006
P. Rahul	RA2411003050029

Project status: proposed design for faculty review. Implementation, testing and deployment evidence are to be completed by the team.

1. Project overview

This project proposes a web application for reporting and resolving campus facility and IT issues. Students and staff submit a request, track its status, and receive updates; authorised helpdesk staff triage, assign, and close requests. A shared relational database replaces scattered messages and registers with searchable records and a simple operational dashboard.

Problem statement. Campus maintenance and IT requests can be lost across paper records, chat messages, and spreadsheets. The proposed system provides one validated, trackable workflow from submission through resolution, with role-based access and an auditable status history.

The first release is intentionally bounded to request intake, triage, assignment, status updates, comments, search, and summary counts. It excludes payment, public access to student data, and integrations with production identity or notification systems.

Target users

USER	NEED
Requester (student or staff)	Submit a request, attach a location/category, view only their own requests, and follow progress.
Helpdesk staff	Review incoming requests, assign a handler, change status/priority, add a work note, and close or reopen.
Administrator	Maintain categories and staff access; review aggregate workload and service history.

Success criteria

A requester can submit a valid ticket and see its generated ID; authorised staff can update it and record a status change; users can search or filter tickets within their access scope; and the dashboard reports counts by status and category. These are acceptance targets, not measured results.

2. Scope and functional requirements

The team will confirm the problem scope and role assumptions with course faculty before implementation. The demo uses synthetic records and does not rely on real sensitive student data.

ID	REQUIREMENT	ACCEPTANCE CHECK
FR-01	Create request with category, location, title and description.	Required fields are validated in React and Spring Boot; successful creation returns a ticket ID.
FR-02	View request list and ticket detail.	List is paginated or bounded; requesters see only their records, staff see their authorised queue.
FR-03	Update status and assignment.	Staff-only action; legal transitions are enforced by the service layer and recorded in history.
FR-04	Edit or withdraw an eligible request.	Requester may edit/withdraw only while New; staff may reopen a resolved ticket with a reason.
FR-05	Search and filter.	Filter by ticket ID, category, status and date; results respect access scope.
FR-06	Add comments/work notes.	Public update and internal note are distinguished; internal notes are staff-only.
FR-07	Show operational summary.	Counts by status/category are derived from persisted records and do not expose requester details.
FR-08	Notify on important status changes.	Notification request is sent through a separate API boundary; local demo records delivery intent.

Non-functional requirements

- Responsive layout with clear navigation, labels, keyboard focus, and readable status indicators.
- Server-side validation and consistent JSON errors; no database credentials or tokens committed to source control.
- Authorisation is enforced by the backend on every protected operation; UI hiding alone is not access control.
- Use synthetic or authorised demo records; avoid public exposure of personal contact details.
- Provide a reproducible local run using environment variables and documented setup.

Out of scope for the initial demo

Live campus account login, real email/SMS delivery, file uploads, service-level analytics based on operational data, and a public production deployment. Any reduction or change in scope will be agreed with faculty.

3. User interface plan

Five views cover the minimum four-view requirement while keeping the workflow straightforward.

VIEW	CONTENTS AND PRIMARY ACTION
1. Home / dashboard	Status and category totals, recent tickets appropriate to the signed-in role, and a prominent “New request” action.
2. Request list	Search field, status/category filters, pagination, ticket ID, title, category, created date and status.
3. Request detail	Description, location, timeline of status changes, comments, current assignee and permitted actions.
4. Create / edit request	Accessible form with required-field feedback, location, category, title, description and confirmation.
5. Staff queue	Unassigned and active tickets, priority, assignment control, status transition, internal work notes.

Interaction and visual rules

- React Router provides named routes; reusable components handle navigation, form fields, status badges, empty states, and error notices.
- UseState manages form and filter state; useEffect loads route data and refreshes the relevant view after successful operations.
- Show field-level validation and a clear success or failure message; preserve entered text when the API rejects a request.
- Responsive CSS uses a consistent spacing scale, high-contrast navy/blue accents, visible focus states, and status labels that do not depend on color alone.

Wireframe summary

Dashboard: top navigation and role indicator → status summary cards → recent requests table → New Request button.
Request detail: ticket ID and status → issue summary and location → chronological activity → permitted comment/status controls.
Staff queue: filter bar → prioritised request rows → assignment and status controls.

4. Architecture and technology

The system follows a three-layer Spring Boot design: REST controllers handle HTTP and input contracts; services enforce workflow rules and access checks; domain entities and repositories persist data with Spring Data JPA. React communicates with the API using JSON. JDBC and SQL concepts are demonstrated through the relational schema, generated SQL inspection, and selected parameterised reporting queries.

USER INTERFACE React + Router Home | Ticket list | Ticket detail | New/edit form | Dashboard	**OPERATIONS API** Spring Boot Controllers | Services | Validation | JPA	**RELATIONAL DATA** MySQL Users | Categories | Tickets | Status history
HTTP / JSON Fetch API, field errors, loading and success states	**SEPARATE BOUNDARY** Notification service REST API; demo adapter / local mode	**QUALITY + DELIVERY** JUnit / MockMvc / React tests Docker Compose local setup

Technology choices

LAYER	PROPOSED TECHNOLOGY	RESPONSIBILITY
Frontend	React, JavaScript, React Router, CSS	Responsive views, forms, routing, client validation and API integration.
Main API	Java, Spring Boot, Bean Validation, Spring Data JPA	Role-aware REST endpoints, business rules, persistence and structured errors.
Database	MySQL 8 relational database	Users, categories, tickets, comments and status history with foreign keys.
Separate service boundary	Notification service REST API	Accepts a notification event and returns an acknowledgement; local demo adapter avoids real delivery.
Quality / local deployment	JUnit 5, MockMvc, React Testing Library, Docker Compose	Automated checks and reproducible local startup; final versions to be selected by the team.

Separate service boundary

The Notification Service is a small, separately runnable REST component. The main application sends an event containing ticket ID, event type, and recipient user ID; the notification component validates the event and records or logs a delivery intent. Its local profile uses a no-op/demo adapter, so no real personal message is sent. The UI obtains notification outcomes through the main API; it does not call internal services directly. This boundary is appropriate to the course requirement without introducing unnecessary microservice complexity.

5. Database schema and relationships

The relational model stores ticket workflow separately from user and category details. All identifiers are generated primary keys; foreign keys preserve referential integrity. Database constraints supplement application validation.

ENTITY	KEY FIELDS / CONSTRAINTS
users	user_id PK; display_name; email UNIQUE; role CHECK (REQUESTER, STAFF, ADMIN); active BOOLEAN. Authentication is a local demo role until faculty approves an identity approach.
categories	category_id PK; name UNIQUE; description; active BOOLEAN.
tickets	ticket_id PK; requester_id FK users; assignee_id FK users NULL; category_id FK categories; title; description; location; priority; status; created_at; updated_at; version.
ticket_comments	comment_id PK; ticket_id FK tickets; author_id FK users; body; visibility (PUBLIC, INTERNAL); created_at.
ticket_status_history	history_id PK; ticket_id FK tickets; changed_by FK users; from_status; to_status; reason; changed_at.

Relationships

- One user can request many tickets; a staff user can be assigned many tickets.
- One category groups many tickets.
- One ticket has many comments and many status-history entries.

Representative SQL queries

The following examples show the intended join and filtered dashboard query. Parameters are bound through prepared statements or repository query methods; values must not be concatenated into SQL.

```
SELECT t.ticket_id, t.title, c.name AS category, t.status,
u.display_name AS requester FROM tickets t
JOIN categories c ON c.category_id = t.category_id
JOIN users u ON u.user_id = t.requester_id
WHERE t.status = ? AND t.created_at >= ?
ORDER BY t.created_at DESC;
SELECT status, COUNT(*) AS ticket_count
FROM tickets GROUP BY status ORDER BY status;
```

Data handling

Deletion is restricted to eligible New tickets and is implemented as a controlled withdrawal; workflow history is retained for traceability. The final schema and retention rules must be reviewed by the team and faculty before any real data is used.

6. API design and examples

Base path: **/api/v1**. JSON responses use consistent resource fields. Protected routes require a server-verified requester/staff/admin role. The precise authentication mechanism is a design decision to confirm before implementation.

METHOD	PATH	PURPOSE / ACCESS
GET	/tickets?status=&categoryId=&q=&page=	List/search; requester sees own tickets, staff queue follows role policy.
POST	/tickets	Create as authenticated requester.
GET	/tickets/{id}	Read detail if caller is requester or authorised staff.
PUT	/tickets/{id}	Edit permitted fields while New; requester or staff by policy.
DELETE	/tickets/{id}	Withdraw eligible New ticket; returns no content.
PATCH	/tickets/{id}/status	Staff changes status, assignment or priority with reason.
POST	/tickets/{id}/comments	Add public comment or staff-only note.
GET	/dashboard/summary	Role-scoped aggregate counts.
POST	/notifications/events	Separate service endpoint; internal/demo use only.

Create request

Request `POST /api/v1/tickets`

```
{"categoryId": 2, "title": "Projector not powering on", "description": "Room display is unavailable before class.", "location": "Block B, Room 204"}
```

Response: 201 Created

```
{"ticketId": 1042, "status": "NEW", "createdAt": "2026-11-09T10:30:00Z"}
```

Validation error

Response: 400 Bad Request

```
{"code": "VALIDATION_ERROR", "message": "Please correct the highlighted fields.", "fieldErrors": {"title": "Title is required"}}
```

Status update

Request `PATCH /api/v1/tickets/1042/status`

```
{"status": "IN_PROGRESS", "assigneeId": 17, "reason": "Assigned to facilities team"}
```

The final implementation should document the agreed transition matrix, authentication scheme, pagination limits and error codes in the README.

7. Business rules and access control

RULE	ENFORCEMENT
Required fields	Title 5-100 characters; description 10-2000 characters; location and active category required. Trim input and reject blank text on client and server.
Status lifecycle	NEW → TRIAGED → IN_PROGRESS → RESOLVED → CLOSED. Staff may return RESOLVED to IN_PROGRESS with a reason; invalid transitions receive 409 Conflict.
Ownership	Requesters can read and edit only their own requests; edits are limited to NEW state. The backend enforces this on every route.
Staff operations	Only staff/admin can assign, change priority/status, and add INTERNAL notes. Public comments remain visible to the requester.
Audit history	Every status change records actor, prior/new value, timestamp and reason in the same transaction as the ticket update.
Deletion	Requester withdrawal is permitted only in NEW state; retain an audit event or use a soft-delete flag.
Concurrency	Use an entity version field or conditional update to reject stale staff edits with 409 Conflict.

Security and privacy

- Use backend identity and access checks; do not trust role, requester ID, or assignee ID supplied by the browser.
- Use parameter binding, allowlisted sort/filter fields, and server-side page-size limits.
- Keep environment secrets outside source control; include only a .env.example with placeholder names, never real values.
- Use synthetic demo identities and avoid logging full descriptions or personal data unnecessarily.
- Configure development CORS narrowly and use HTTPS for any eventual public deployment.

8. Test plan and evidence to collect

Testing is planned; no test execution has been claimed in this proposal. The team should capture results after implementation and include them in the final package.

TEST LEVEL	EXAMPLE CHECKS	EVIDENCE
React component	Required form fields; validation message; status badge; empty/error/loading states.	Test output and screenshots of the relevant states.
Service unit tests	Allowed/blocked status transitions; requester ownership; close/reopen rule; assignment policy.	JUnit report with pass/fail totals and any fixes.
Controller/API tests	Create valid ticket (201); invalid input (400); missing record (404); forbidden access (403); invalid transition (409).	MockMvc request/response tests.
Repository/integration	Persist and retrieve linked ticket/category/user; verify join/report query and transaction history.	Test database output or CI report.
End-to-end demo	Requester creates ticket; staff assigns and advances it; requester sees progress; filter and summary update.	Dated screenshots or short demo recording plus reproducible seed data.

Test data should be resettable, synthetic and small enough to inspect. Record the software versions and commands used, the test date, pass/fail counts, and unresolved defects. Do not label an unrun test as passed.

9. Deployment and local run plan

A local Docker Compose setup is the reproducible minimum deliverable. A public deployment URL is optional and should only be added after the team has an approved host, secure configuration and a successful smoke test.

STEP	PLANNED INSTRUCTIONS
Prerequisites	JDK 21 (or the version selected by the team), Node.js LTS, Docker Desktop / Docker Engine, Git. Record exact versions in README.
Configuration	Copy .env.example to .env and set DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, API_PORT and other required local values. Keep .env out of Git.
Database	Docker Compose starts MySQL with a named volume and a health check. Apply versioned schema migrations and load synthetic seed data.
Backend	Run the Spring Boot application with the local profile and configured database environment. Document the exact Gradle/Maven wrapper command after project generation.
Notification API	Start the notification component with a demo/no-op delivery adapter and configure its URL in the backend environment.
Frontend	Install from the lockfile, set the API base URL, and start the React development server; document production build/serve steps.
Verification	Check health endpoint, create a seeded test request, exercise a staff transition, verify database persistence, and capture test/build output.
Deployment	If deployed, use managed secrets, HTTPS, least-privilege database access, and a smoke test. Add the live URL only after access and behaviour are confirmed.

Planned repository layout

```
campus-helpdesk/
frontend/ | backend/helpdesk-api/ | services/notification-service/
database/migrations/ | database/seed/ | docs/architecture/ | docker-compose.yml |
README.md
```

A complete README should include prerequisites, setup, environment variable names, schema and seed instructions, build/run commands, endpoint examples, test commands, demo roles, known limitations and verified deployment details.

10. Work plan and course outcome mapping

MILESTONE	WORK	DELIVERABLE / CHECKPOINT
1. Proposal	Confirm problem, target users, boundaries, user stories and faculty approval.	Approved problem statement and responsibilities.
2. Design	Produce wireframes, architecture diagram, schema/ER diagram, endpoint plan and transition rules.	Reviewed design package.
3. Implementation	Build React views, Spring Boot layers, SQL/JPA persistence and notification API.	Working CRUD path across at least two related entities.
4. Integration and tests	Connect UI/API/database; implement validation, access checks, error handling and tests.	Test output, issue log, demonstration checklist.
5. Deployment and demo	Package local setup, prepare synthetic records, capture screenshots and explain member contributions.	README, reproducible demo and final presentation/report.

Suggested team responsibilities

MEMBER	PROPOSED FOCUS
S. Nithish	Coordinator, requirements and integration tracking; confirm API/UI handoffs.
P. Karthik	React views, routing, responsive styling and client-side validation.
S.M.P. Deekshitulu	Spring Boot controllers, service rules, API contracts and backend tests.
P. Rahul	Relational schema/JPA, seed data, integration quality, README and local deployment plan.

These are starting assignments, not exclusive ownership. All four members should review the architecture, run the application, understand the complete request lifecycle, and contribute to the demonstration. Replace or rebalance roles if the team agrees.

Course outcome mapping

CO	TASK EVIDENCE
CO-1	Semantic HTML, CSS layout, navigation, accessible forms/tables and responsive pages.
CO-2	JavaScript and React components, events, forms, routing, hooks and client-side validation.
CO-3	Relational design, SQL CRUD and joins, parameterised reporting queries, JDBC/database connectivity concepts.
CO-4	Spring Boot setup, controller/service/domain layers, business rules and tests.
CO-5	JPA entities/repositories plus a separately runnable notification API boundary integrated through the backend.

11. Demonstration and submission checklist

Demonstration flow

1. Open the dashboard and explain requester versus staff access.
2. Submit a request with invalid fields to show validation, then create a valid synthetic request.
3. Find it in the staff queue, assign it, and move it through the allowed workflow with a reason.
4. Return to requester view and show the status timeline and permitted public updates.
5. Filter/search the request list and show aggregate counts.
6. Explain the related database records, service/API boundary, tests and local deployment commands.

Package checklist

- Faculty-approved problem statement, users, scope and functional requirements.
- Architecture diagram, UI wireframes, ER diagram and schema/migration scripts.
- React frontend, Spring Boot backend and notification service source code.
- Synthetic seed dataset, API endpoint list and request/response examples.
- Test plan, actual test results, issue log and representative screenshots/demo recording.
- Reproducible deployment instructions, environment variable template and README.
- Individual contribution statement from each member; all members prepare to explain the full system.

Individual contribution statement template

TEAM MEMBER	CONTRIBUTION / EVIDENCE TO COMPLETE
S. Nithish	Record implemented tasks, commits/design evidence, integration work and demo explanation.
P. Karthik	Record implemented views/components, validation, tests and demo explanation.
S.M.P. Deekshitulu	Record backend/API rules, tests and demo explanation.
P. Rahul	Record schema/persistence, integration/deployment documentation and demo explanation.

Before submission: confirm faculty approval, replace planned items with implemented evidence, run and record the tests, verify the local setup from a clean start, and ensure every team member can explain the complete system.

Source basis: Task 4 - Team Project Assignment, 21CSE354T Full Stack Web Development (course brief supplied for this assignment).